

Project Report: Predicting Analyte Retention Times

Solène Noize, Melina Cherchali

Abstract—This project revolves around the relationship between retention time (RT) and the chemical properties of drugs within the context of liquid chromatography setups in diverse laboratories. Our objective is to develop models to predict RT of chemicals using machine learning techniques. This report provides an overview of our methodology, describing our data preprocessing approach, as well as the exploration and evaluation of diverse models.

I. INTRODUCTION

The objective of this project is to implement predictive models capable of determining the RT of analytes based on various chemical features. The training data set is composed of 3500 known chemicals, their corresponding RT, the laboratory from which the sample comes from, as well as their Extended-Connectivity Fingerprints (ECFPs) and simplified molecular-input line-entry system (SMILES). The name of the compound, and molecular representation (mol), are also features of the data.

II. DATA PREPROCESSING

During a first preprocessing step, missing values were removed from the data, as well as correlated and constant columns. In fact, as seen on Figure 1, numerous columns show limited variance, which is why we exclude those exhibiting a standard deviation inferior to 90%. No standardizing was needed for the ECFP columns, as the data is binary. We dropped the features containing the name of the compound and molecular representations, as the SMILES already provided the information. We used the RDKit package, specifically modules such as Descriptors and ChemicalFeatures, in order to extract additional predictors from the SMILES column such as the number of atoms, bonds, rings, functional groups and more. We also incorporated the option to include the CDDD columns in the training dataset, but our analysis indicated that this leads to an increase in the test error. We finally proceeded to use a one-hot encoding approach to convert our laboratory predictor to numerical values.

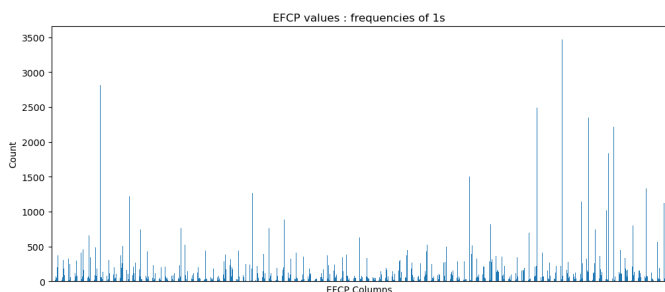


Fig. 1: ECFP columns distribution

We visualised the retention time distribution in order to get a better sense of the overall pattern and variability within the dataset. As represented in Figure 2, we observed a rightward skew, suggesting a prevalence of lower RT values within the dataset.

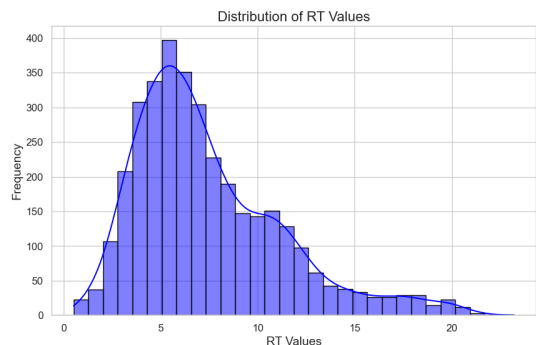


Fig. 2: Retention times distribution

III. METRICS

To evaluate the models implemented in this project, we used the root mean square error (RMSE) as a metrics, and try to minimize it in order to enhance the accuracy of our predictions.

We split our data, using 80% for training the models and 20% for testing, and applied a shuffling technique to ensure an unbiased representation during the training process.

IV. LINEAR MODELS

A. Linear Regression

As a first approach, we trained a Linear Regression on the training data. As depicted on Table 1, the training error was high, indicating that the model is not suited at capturing the complex patterns within the dataset.

We tried both Lasso and Ridge regularization, fine tuning the hyperparameters with a Grid Search, and using Cross Validation to reduce overfitting.

Additionally, due to the high dimensionality of our dataset, we opted to employ Principal Component Analysis (PCA) as a preprocessing step before applying linear models. This decision was made to diminish the dimensionality of the feature space, mitigate noise in the dataset, and enhance the model's performance by transforming features into a set of linearly uncorrelated variables.

Nevertheless, as depicted in Table 1, the RMSE remained unsatisfactory. In fact, linear models, while effective in various scenarios, may prove inadequate when confronted with intricate patterns, and non-linearities of real-world datasets. This drove

us to explore and incorporate non-linear models into our methodology.

Models	CV error		Test error	
	PCA	no PCA	PCA	no PCA
Linear	1.30	2.13	1.30	1.55
Ridge	1.30	1.53	1.30	1.40
Lasso	1.27	1.21	1.29	1.23

TABLE I: Comparison of RMSE scores for linear models

V. NON LINEAR MODELS

Non-linear models present the capacity to discern more nuanced relationships, offering a flexible framework to navigate the intricacies of our data. Unlike with the linear approach, we do not employ PCA on the data. In fact, non-linear models possess the ability to operate effectively in high-dimensional spaces, making them well-suited for scenarios where the relationships between variables are complex.

A. Support Vector Machines

As a first approach, we chose to implement a Support Vector Machines (SVM) model due to its resilience to overfitting, particularly in high-dimensional spaces. Yet, upon training our dataset with this model and subjecting it to a 5-fold Cross Validation, the results unveiled a suboptimal performance, even falling behind some of the linear models we had introduced previously.

B. Decision Trees

Subsequently, we proceeded to implement the Random Forest model. The RMSE was better than with SVM but still not satisfactory enough. We then decided to employ a Gradient Boosting model, known for its predictive accuracy, and proficiency in discerning non-linear patterns. Furthermore, the algorithm leverages gradient descent optimization to minimize the loss function during the training phase. Remarkably, after tuning through Grid Search and employing Cross Validation to mitigate overfitting, this model emerged as our most effective choice, as we can see on Table 2.

C. Neural Network

In an attempt to achieve even better predictions, we moved on to Neural Networks. We explored the PyTorch and Keras frameworks. For both, we tuned the hyperparameters and trained our model, as done with the previous methods. The results were quite encouraging, reaching even lower test errors than with Gradient Boosting.

PyTorch Neural Network : Our model comprised two hidden layers, each containing 220 neurons. Additionally, we incorporated a dropout rate of 0.2 to enhance the generalization capability and prevent overfitting. To further ensure robust training, we employed early stopping with a patience of 25 epochs to halt training if no significant improvements in performance within this period.

Keras Neural Network : The Keras framework was employed to construct a three-branch network. Each contains

multiple layers, with various number of neurons, allowing the model to learn complex representations. Similar to the PyTorch model, the Keras Neural Network incorporated early stopping with a patience of 15 epochs to prevent overfitting.

Non-linear models	CV error	Test error
SVM	2.08	2.08
Random Forest	1.03	0.95
Gradient Boosting	0.78	0.67
Pytorch neural network	0.85	0.72
Keras neural network	0.82	0.71
Ensemble model	0.86	0.76

TABLE II: Comparison of RMSE scores for non-linear models

VI. MODEL ASSEMBLING

Having explored various models, we aimed to leverage the strengths of both Gradient Boosting and Neural Networks by merging them into an ensemble model. Ensemble models often demonstrate superior predictive performance by combining the strengths of individual models.

A. Gradient Boosting and Neural Network Ensemble

To construct our ensemble model, we trained both the Gradient Boosting and Neural Network models (PyTorch and Keras) on the training data. The predictions from these three models were then averaged to create the final ensemble prediction.

The rationale behind this combination lies in the complementary nature of the models. Gradient Boosting excels in capturing complex non-linear patterns, and Neural Networks are powerful in learning intricate relationships within the data. Pytorch and Keras each offer unique perspectives due to differences in framework, architecture, and training methodologies.

By combining these models, we aimed to benefit from the non-linear pattern recognition capabilities of these algorithms. This approach has the potential to improve generalization performance and enhance predictive accuracy compared to using each model individually.

This method showed the best results out of all models implemented, and we used cross validation to ensure no overfitting occurred. The results of this ensemble approach are presented in Table 2.

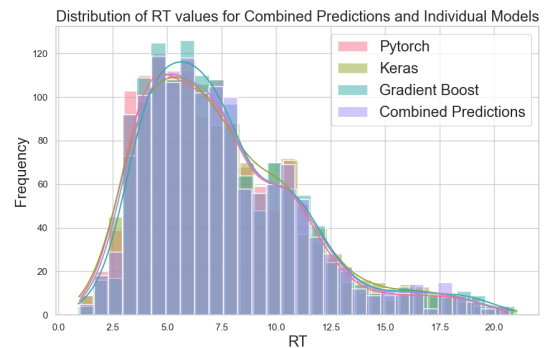


Fig. 3: Ensemble method: prediction distribution